

BANK OF TANZANIA

AI Innovation Observatory

for Central Banking & Financial Sector Intelligence



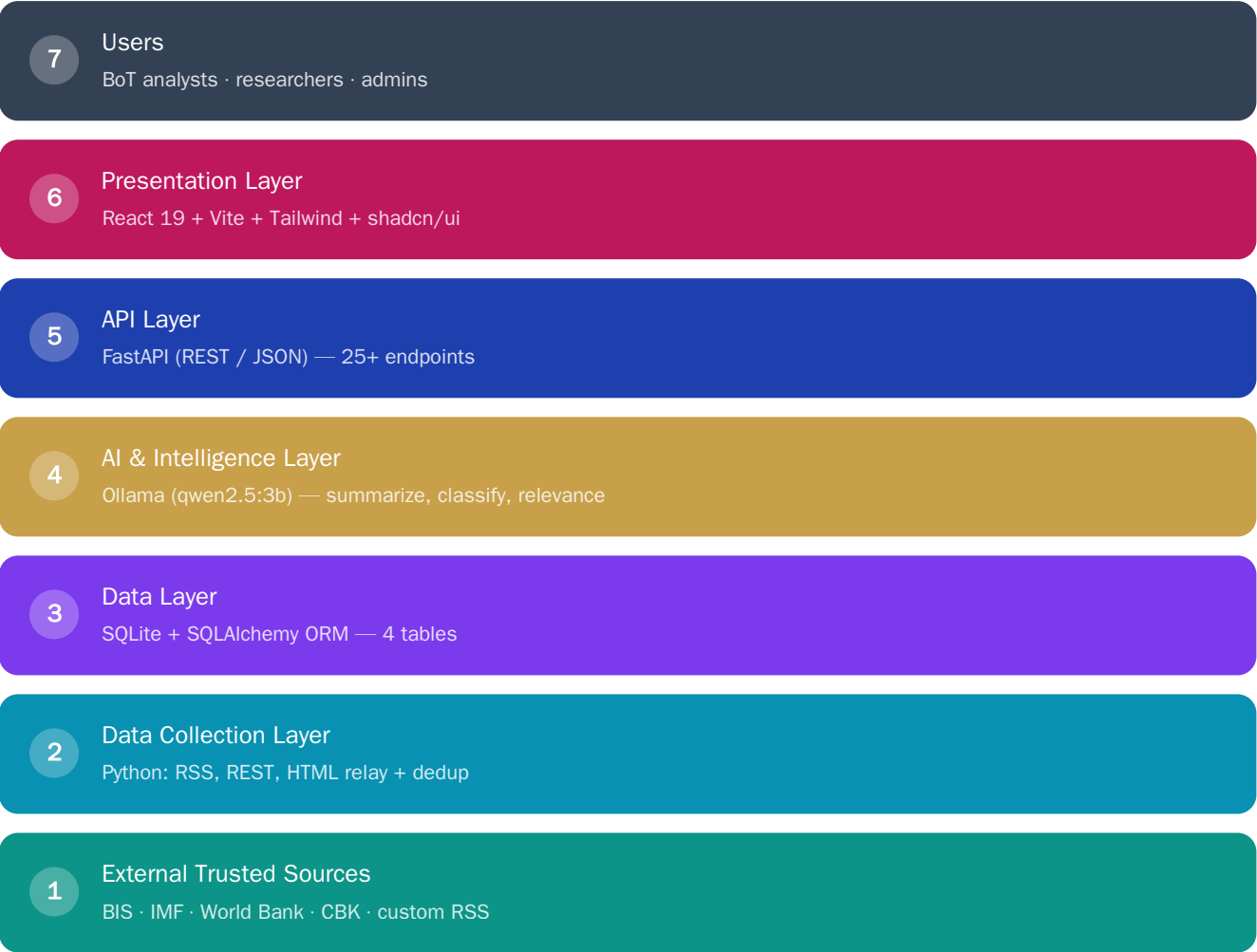
System Architecture & Database Design

Student Project Prototype

September 2026

1. System Architecture — 7 Layers

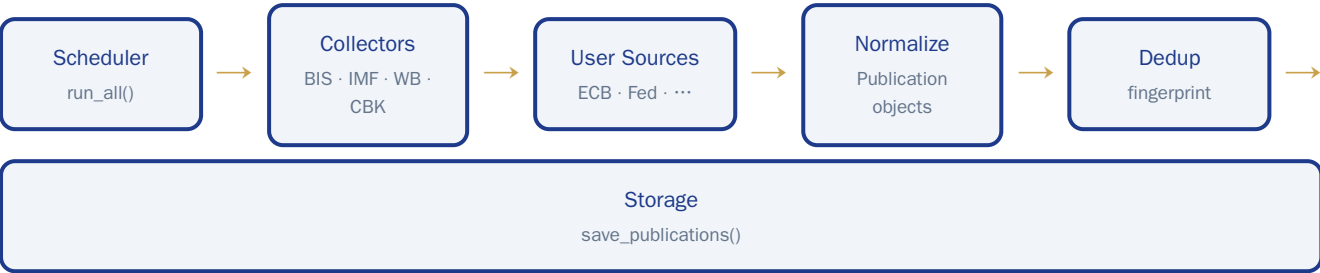
Bottom-up: data flows from external sources through collection, storage, and AI up to the analyst dashboard.



Design principle: Each layer only talks to the layer directly above and below it. Changing the AI model does not affect the API. Adding a source does not affect the frontend.

2. Data Collection Layer

Fetches from 4 built-in sources + any user-added RSS feed. Normalizes everything into one schema.



Built-in collectors

Source	Method	Endpoint	Pubs/run
BIS	RSS × 3	bis.org/doclist/*.rss	~70
IMF	HTML relay	rss-parrot.net (mirror of IMF)	~13
World Bank	REST API	search.worldbank.org/api/v3/wds	~55
CBK	RSS	centralbank.go.ke/feed	~10

Collection controls

Error isolation

Each collector wrapped in `safe_fetch()` . One source failing does not stop others.

Duplicate detection

SHA-256 of `institution|source_url|title` — UNIQUE constraint in DB.

Dynamic sources

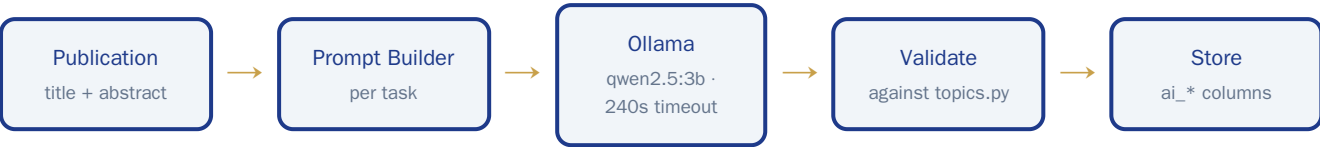
Admins add any RSS URL via the UI — validated before saving.

Scheduling

Manual (admin button) or automatic (APScheduler, 15 min – 6 hr interval).

3. AI & Intelligence Layer

Local LLM produces structured intelligence for every publication. No external API. No cost.



Three AI tasks per publication

Task	Input	Output
Summarize	title + 800 chars of abstract	6-section summary (topic, purpose, findings, developments, policy, relevance)
Classify	title + abstract	1–3 slugs from 11 canonical topics
Relevance	title + abstract	2–3 sentences on BoT relevance + disclaimer

Three safeguards

Fallback engine

If Ollama is down → rule-based keyword matching. Pipeline never stalls.

Topic validation

LLM output is validated against `topics.py` . Hallucinated slugs dropped.

Never empty

Institution-aware default topic — every processed publication ends with at least 1 topic.

Traceability

`ai_engine` column records "ollama" or "rule-based" for every row.

Model choice: qwen2.5:3b — small, local, CPU-friendly. Zero API cost, data never leaves the machine. Accuracy ~65–75% top-1 on the manual eval sample.

4. API & Presentation Layers

API Layer — FastAPI

Stateless REST backend. Only the frontend calls this. 25+ endpoints grouped by resource.

Group	Endpoints
Publications	<code>GET /publications</code> , <code>GET /publications/{id}</code> , filters by source/topic/date
Search	<code>GET /search?q=...</code> across title + abstract + AI summary
Trends	<code>/trends</code> , <code>/trends/emerging</code> , <code>/trends/timeline</code> , <code>/trends/insights</code> , <code>/trends/glance</code>
Stats	<code>/stats</code> , <code>/stats/timeline</code> with hidden/orphaned counts
Admin	<code>/admin/collect</code> , <code>/admin/logs</code> , <code>/admin/sources</code> , <code>/admin/rules</code> , <code>/admin/alerts</code>

Presentation Layer — React

Page	Purpose	Role
Dashboard	Overview + AI glance + high-relevance	Analyst
Publications	Browse + filter + sort + detail drawer + CSV export	Analyst
Trends & Insights	Multi-color line chart + AI narrative	Analyst
Topic Classification	11 topic tiles → drawer of publications	Analyst
Search & Discover	Keyword search + query insights	Analyst
Knowledge Base	Browse by topic / source / time	Analyst
Alerts	Live system alerts + rule builder	Both
Admin (6 pages)	Sources · Publications · Topics · Users · Alerts · Dashboard	Admin

Communication: Frontend calls `/api/*` → Vite proxies to backend at `http://backend:8000` inside Docker. No CORS. No hardcoded URLs.

5. Database Design — Entity Relationship

SQLite + SQLAlchemy 2.0. Four tables. File: `backend/observatory.db`

publications	
id	INT PK
title	STR
institution	STR·IDX
source_url	STR
published_date	DT·IDX
document_type	STR
abstract	TEXT
raw_content	TEXT
fingerprint	STR·UNIQ
ai_summary	TEXT
ai_topics	STR
ai_relevance	TEXT
ai_processed	BOOL
ai_engine	STR
hidden	BOOL·IDX
manual	BOOL
collected_at	DT

id	INT PK
name	STR-UNIQ
url	STR
method	STR
active	BOOL
builtin	BOOL
added_by	STR
added_at	DT

🔔 alert_rules		
id		INT PK
keyword		STR
source		STR
topic		STR
priority		STR
created_by		STR
created_at		DT
active		BOOL

rule_matches	
id	INT PK
rule_id	INT-IDX
publication_id	INT-IDX
matched_at	DT

Relationships: Logical (dotted), not enforced FKs. Sources produce publications. Rules match publications via the `rule_matches` bridge table.

6. Database — Design Decisions

Why these choices

Decision	Reason
SQLite over PostgreSQL	Zero-setup, single file, portable. Swap by changing one env var.
Fingerprint UNIQUE	DB enforces dedup — even if code has a bug, no duplicate rows.
AI columns nullable	Publication stored first; enriched second. Pipeline never blocks on AI.
Topics as comma-separated strings	Fast, simple, good enough. Upgrade path: join table.
hidden instead of delete	Non-destructive. Admins can restore. Traceability preserved.
Source table separate from built-ins	Built-ins have custom parsers. Only dynamic feeds go in the table.
Rule matches stored, not computed	Avoids re-scanning all pubs every time alerts are viewed.

Known limitations

Limitation	Impact	Fix path
Users in memory, not DB	Logins reset on restart	Add User model + hashed passwords
ai_topics as string	Harder SQL joins	Add publication_topics M2M
No FK constraints	Orphan rows possible	Add ON DELETE CASCADE
SQLite write concurrency	Locks under heavy load	Switch to PostgreSQL

Sample queries

Count by source: `SELECT institution, COUNT(*) FROM publications WHERE hidden=0 GROUP BY institution;`

Fallback rows: `SELECT id, title FROM publications WHERE ai_engine='rule-based';`

Recent rule matches: `SELECT r.keyword, p.title FROM rule_matches m JOIN alert_rules r ON r.id=m.rule_id JOIN publications p ON p.id=m.publication_id ORDER BY m.matched_at DESC LIMIT 10;`

Traceability guarantee: Every AI-generated output stores its engine and links back to `source_url` — the original document can always be verified.